

A Let's Read and Find Out Book™

How to Talk to Your **COMPUTER**

By SEYMOUR SIMON



Illustrated by Barbara and Ed Emberley

Programming Model or Architecture

Before a program is written, the internal architecture of microprocessor must be known.

The **programming model**

- program visible
- program invisible

In program visible:

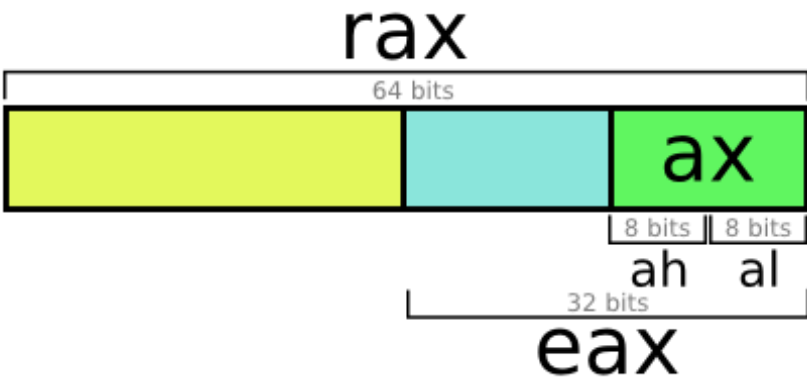
- Registers are used during application programming
- And are specified by the instructions.

But in program invisible:

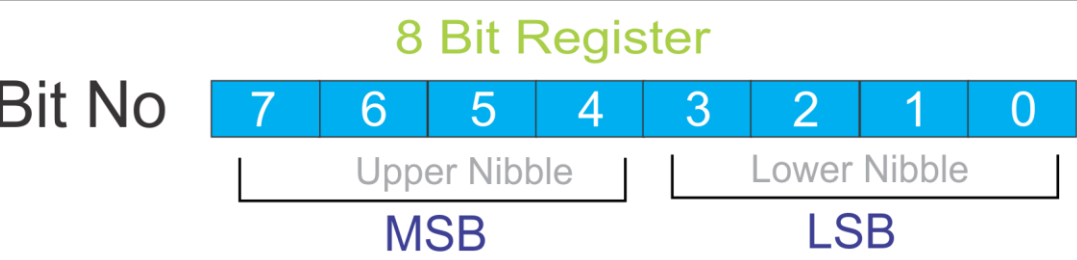
- the registers are not addressable directly during applications programming but may be used indirectly during system programming (control and operate protected memory system and other features).

Programming Model:

First we study, one of the FUs of CPU: Registers



	64-bit Names	32-bit Names	16-bit Names	8-bit Names
RAX		EAX	AX	AH AL
RBX		EBX	BX	BH BL
RCX		ECX	CX	CH CL
RDX		EDX	DX	DH DL
RBP		EBP	BP	
RSI		ESI	SI	
RDI		EDI	DI	
RSP		ESP	SP	
64 bits 32 bits 16 bits				
R8				
R9				
R10				
R11				
R12				
R13				
R14				
R15				
RFLAGS		EFLAGS	FLAGS	
RIP		EIP	IP	



CS
DS
ES
SS
FS
GS

Multipurpose Registers

AH	AL	AX (primary accumulator)
BH	BL	BX (base, accumulator)
CH	CL	CX (counter, accumulator)
DH	DL	DX (accumulator, other functions)
SI		Source Index
DI		Destination Index
BP		Base Pointer

AX: It's a 16-bit register or either used as 8-bit registers (AH or AL). The accumulator is used for instructions such as addition, subtraction, multiplication and division.

BX: It some time **holds** the **offset address** of a location in memory system.

CX: It hold the **count** for various instructions.

DX: It **holds** the part of the **result** from a **multiplication** or part of **dividend before division**.

BP: it **points** to a **memory location** for data transfer.

SI: It often **addresses source string data** for the string instructions.

DI: It often **addresses string destination** data for the string instructions.

Special-Purpose Registers

SP

Stack Pointer

IP

Instruction Pointer

IP: It addresses the next instruction in a section of a memory defined as a code segment.

SP: It addresses an area of memory called stack.

Status Register or Flag Register

13 12 11 10 9 8 7 6 5 4 3 2 1 0 (bit position)

- - O D I T S Z - A - P - C Flags

Flag Register: It indicates the condition of the microprocessor and control its operation. (6-Conditional and 3-Control flags).

C (carry): Carry holds the carry after addition or the borrow after subtraction.

P (parity): Parity is a logic 0 for odd parity and a logic 1 for even parity. Parity is the count of ones in a number expressed as even or odd.

A(auxiliary carry): The AC holds the carry (half-carry) after addition or subtraction between bit positions 3 and 4 of the result.

Z (zero): It shows the result of an arithmetic or logic operation is zero. If Z=1, the result is zero; if Z=0, the result is not zero.

S (sign): It holds the arithmetic sign of the result after as arithmetic or logic instruction executes.

T (trap): It enables trapping through an on-chip debugging feature.

I (interrupt): It controls the operation of INTR (interrupt request) input pin.

D (direction): It selects either the increment or decrement mode for the DI or SI registers during string instructions.

O (overflow): It occurs when **signed numbers are added or subtracted**. An overflow indicates that the result has exceeded the capacity of a machine. **For unsigned operations, the overflow flag is ignored.**

Show how the flag register is affected by the addition of 38H and 2FH.

Solution:

MOV	BH,38H	;BH= 38H	
ADD	BH,2FH	;add 2F to BH, now BH=67H	
	38	0011	1000
+	2F	0010	1111
	67	0110	0111

CF = 0 since there is no carry beyond d7
 PF = 0 since there is an odd number of 1s in the result
 AF = 1 since there is a carry from d3 to d4
 ZF = 0 since the result is not zero
 SF = 0 since d7 of the result is zero

Show how the flag register is affected by

MOV	AX,34F5H	;AX= 34F5H	
ADD	AX,95EBH	;now AX= CAE0H	

Solution:

	34F5	0011	0100	1111	0101
+	95EB	1001	0101	1110	1011
	CAE0	1100	1010	1110	0000

CF = 0 since there is no carry beyond d15
 PF = 0 since there is an odd number of 1s in the lower byte
 AF = 1 since there is a carry from d3 to d4
 ZF = 0 since the result is not zero
 SF = 1 since d15 of the result is one

Show how the flag register is affected by

MOV	AL,9CH	;AL=9CH	
MOV	DH,64H	;DH=64H	
ADD	AL,DH	;now AL=0	

Solution:

	9C	1001	1100
+	64	0110	0100
	00	0000	0000

CF=1 since there is a carry beyond d7
 PF=1 since there is an even number of 1s in the result
 AF=1 since there is a carry from d3 to d4
 ZF=1 since the result is zero
 SF=0 since d7 of the result is zero

Show how the flag register is affected by

MOV	BX,AAAAH	;BX= AAAAH	
ADD	BX,5556H	;now BX= 0000H	

Solution:

	AAAA	1010	1010	1010	1010
+	5556	0101	0101	0101	0110
	0000	0000	0000	0000	0000

CF = 1 since there is a carry beyond d15
 PF = 1 since there is an even number of 1s in the lower byte
 AF = 1 since there is a carry from d3 to d4
 ZF = 1 since the result is zero
 SF = 0 since d15 of the result is zero

Segment register

CS	Code Segment
DS	Data Segment
ES	Extra Segment
SS	Stack Segment

It generates memory address when combined with other registers in a microprocessor. There are either **four** or **six segments registers** in various versions of microprocessor.

CS: The code segment is a section of memory that holds the code (program and procedures) used by the microprocessor. It defines the starting address of the section of memory holding code. In real mode it defines the start of 64k-byte memory.

DS: The data segment is a section of memory that holds the most data used by program.

ES: The extra segment is an additional data segment that is used by some of string instructions to hold destination data.

SS: The stack segment defines the area of memory used for the stack. The stack entry point is determined by the stack segment and stack pointer register. The BP register also address data with in the stack segment.

Real Mode Memory Addressing

Real Mode operation allows the microprocessor to address only the **first 1M byte of memory space**. Only the 8086 and 8088 operate in the real mode. Both have 20bit of address bus.

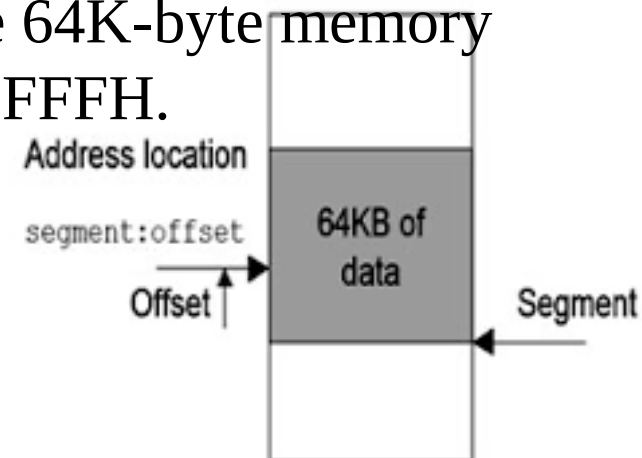
$2^{20} = 1048576 \text{ Bytes} = 1 \text{ MByte}$

Have a range of 00000H-FFFFFFH.

Segments and Offsets: A combination of a segment address and an offset address accesses a memory location in the real mode.

- ✓ **Segment address**, define the beginning of any 64K-byte memory segment.
- ✓ **Offset address** selects any location within the 64K-byte memory segment. Therefore its range from 0000H to FFFFH.

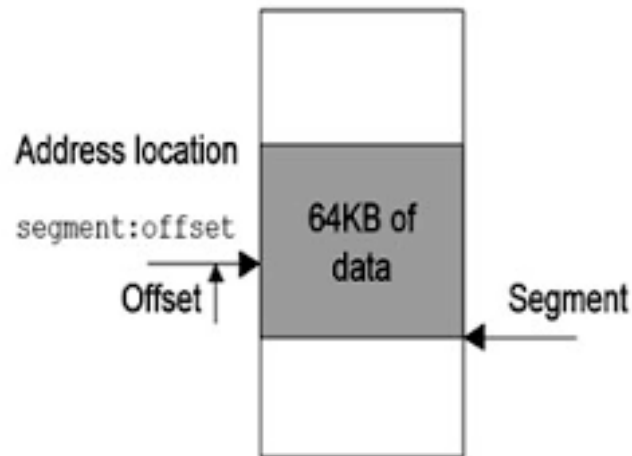
<i>Segment</i>	<i>Offset</i>	<i>Special Purpose</i>
CS	IP	Instruction address
SS	SP or BP	Stack address
DS	BX, DI, SI, an 8 or 16 bit number	Data address
ES	DI for string instructions	Extra address



Logical address and physical address:

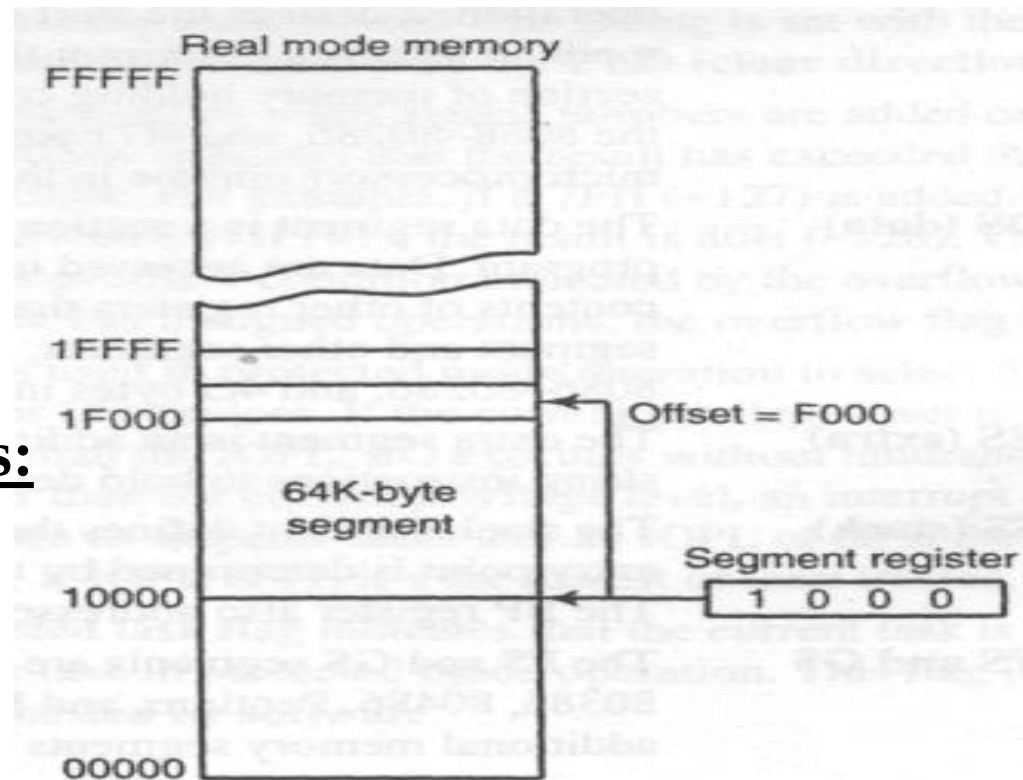
The physical address is the 20-bit address that is actually put on the address pins of the 8086 microprocessor.

The logical address consists of segment value and offset address. [Segment Address : Offset Address]



How to make a physical address:

- i) Start with segment address.
- ii) Append zero to segment address.
- iii) Add offset address.



Starting and Ending Address:

For exp: Segment Address = 1200H its physical address is 12000H (if Offset Address = 0000H). Ending Address is found by adding FFFFH.

For exp: Segment Address : 3000H, first address is 30000H, last address is 30000H+FFFFH or 3FFFFH.

Segment Register	Starting Address	Ending Address
2000H	20000H	2FFFFH
2001H	20010H	3000FH
2100H	21000H	30FFFH
AB00H	AB000H	BAFFFH
1234H	12340H	2233FH

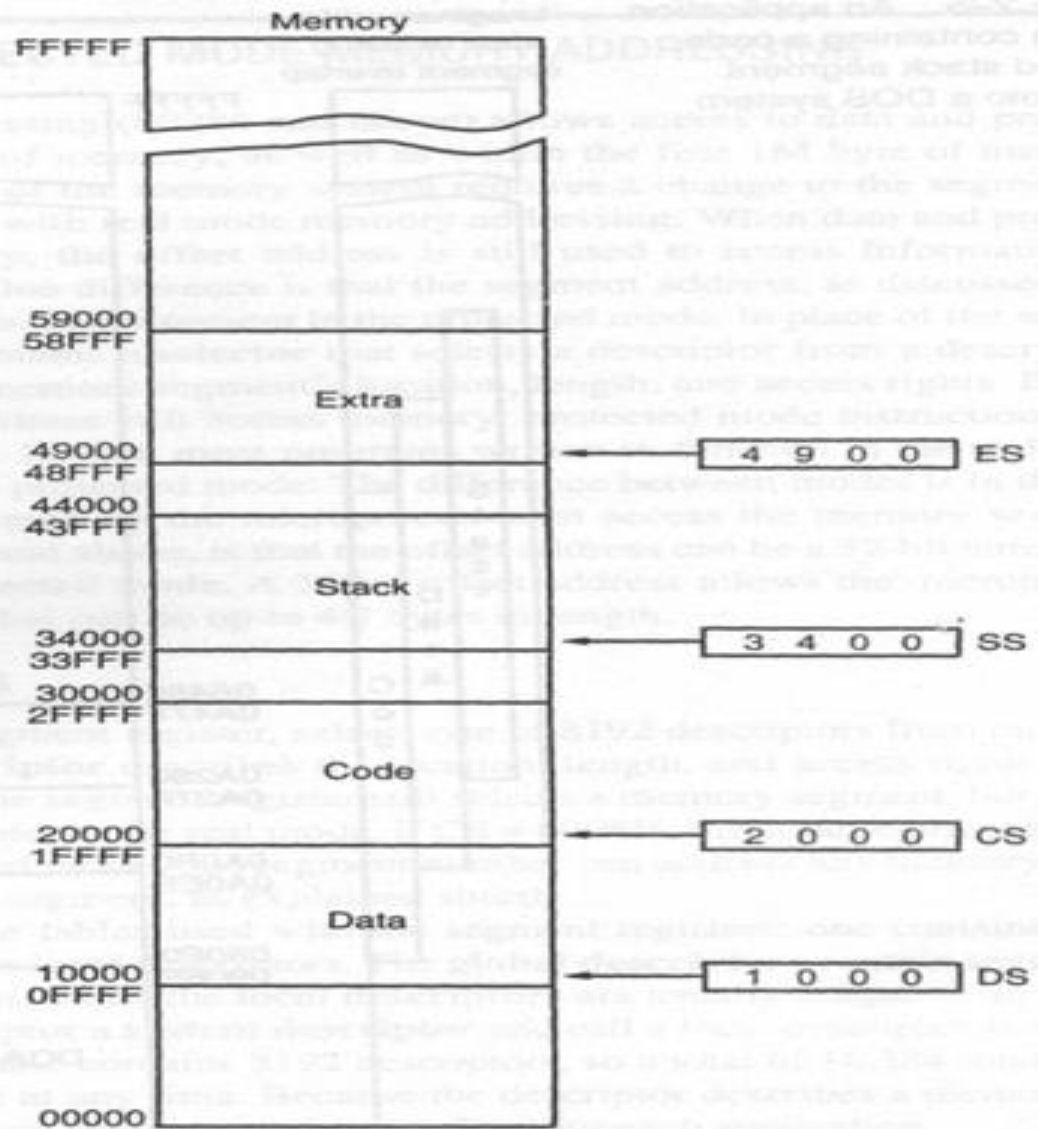


Figure 2-4 A memory system showing the placement of four memory segments.

A memory segment can touch or even overlap if 64K-byte of memory is not required.

Imaginary side
view detailing
segment overlap

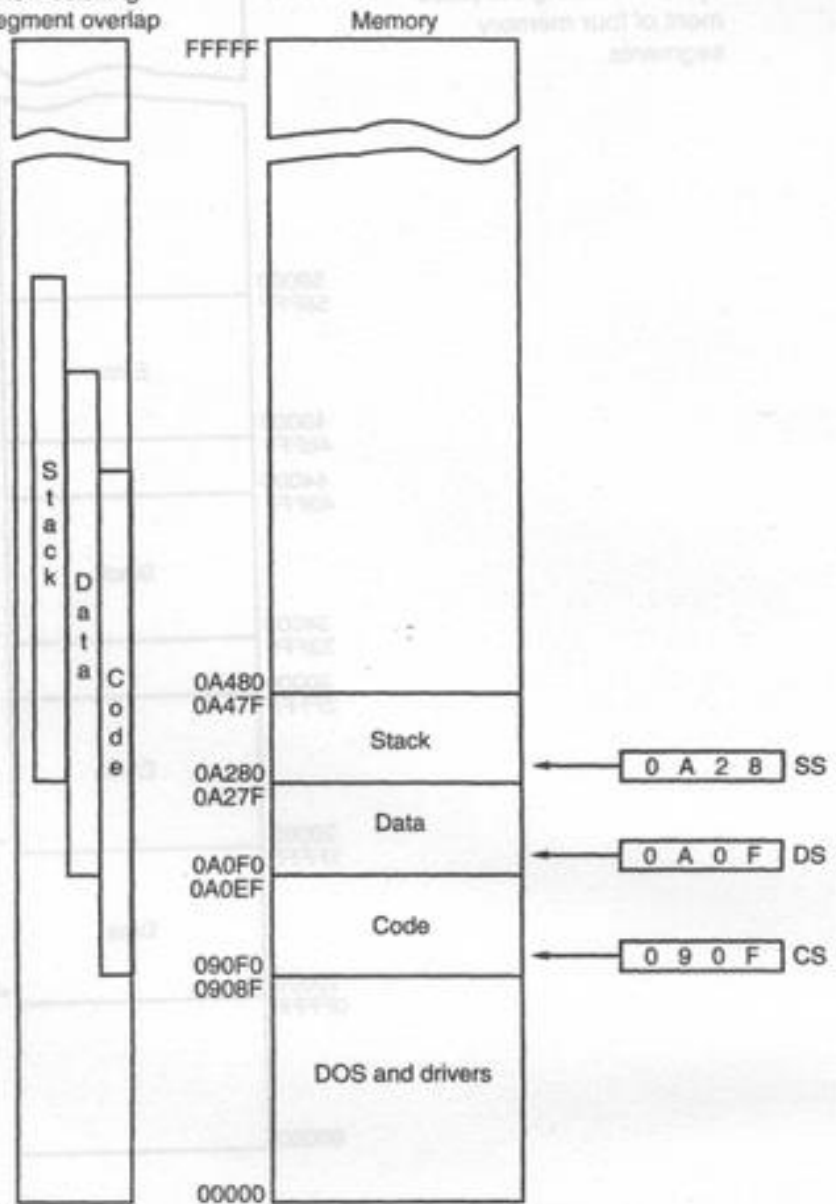


Figure 2-5 An application program containing a code, data, and stack segment loaded into a DOS system memory.

*In the real mode, show how the starting and ending address of each segment located by the following segment register values:

- a) 1000H b) 1234H c) 2300H d) E000H e) AB00H

*Find the memory address of the next instruction executed by the microprocessor, when operated in real mode, for the following CS:IP;

- a) CS: 1000H and IP: 2000H b) CS: 2000H and IP: 1000H
c) CS: 2300H and IP: 1A00H d) CS: 1A00H and IP: B000H
e) CS: 3456H and IP: ABCDH

*Determine the memory location addressed by the following real mode 80286 register combinations;

- a) DS: 1000H and DI: 2000H b) DS: 2000H and SI: 1002H
c) SS: 2300H and BP: 3200H a) DS: A000H and BX: 1000H
e) DS: 2900H and SP: 3A00H

Data-Addressing Modes

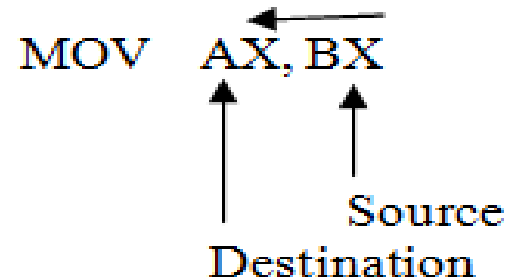
The **MOV instruction** is very common and flexible instruction, it provides a basis for the explanation of the data-addressing modes.

MOV is an opcode (operation code, tells the microprocessor which operation to perform).

The source and destination are often called operands.

The data addressing modes are follows:

- a) Register b) Immediate c) Direct
- d) Register indirect e) Base-plus-index f) Register Relative
- g) Base relative-plus-index



Register Addressing: Register addressing transfers a copy of byte or a word from a source register or contents of a memory location to the destination register or a memory location.

Examples:

MOV AL,BH

MOV SI,DI

MOV CS,DS

MOV CH,CL

MOV BX,EX

MOV BL,DX

MOV CX, DX

MOV DS,CX

MOV CS,AX

Immediate Addressing: Immediate addressing transfers the source, an immediate byte, word of data, into the destination register or memory location.

Examples:

MOV AL,22H	MOV AL,'A'	MOV BX,'ABCD'
MOV AX,44H	MOV AH,1	MOV AX, F1234F
MOV CH,100	MOV AX,'AB'	

Direct Addressing: Direct addressing moves a byte or word between a memory location and a register. The instruction set does not support memory-to-memory transfer, except with the MOVS instruction.

Examples:

MOV [1234H],AX	MOV AL,NUMBER
MOV CX,[9876H]	MOV BX, COW
MOV CX, LIST	(Contents of data segment memory location)

Register Indirect Addressing: Register indirect addressing transfer a byte or word between a register and a memory location addressed by an index or base register. The index and base registers are BP, BX, DI and SI.

Example:

```
MOV [BX], CL
MOV AX, [BX]
```


Base-plus-index Addressing: byte or word between a register and a memory location addressed by a base register (BX or BP) plus an index register (SI or DI).

Examples:

MOV [BX+DI], AX

MOV CH,[BP+SI]

Register Relative Addressing: Register relative addressing moves a byte or word between a register and the memory location addressed by an index or base register plus a displacement (also called effective address).

Example:

MOV AX, [DI+100H]

MOV ARRAY[SI],BL

MOV LIST[SI+4],BL

Base Relative-Plus-Index Addressing:

The base relative-plus-index addressing mode is similar to base-plus-index addressing, but it adds a displacement, besides using a base register and an index register, to form the memory address. This type of addressing mode often addresses a two-dimensional array of memory data.

Example:

MOV DH,[BX+DI+20H]

MOV FILE[BP+SI+4],CL

Assembly Language

Assembly language program consists of four parts or fields

LABEL	OPCODE	OPERAND	COMMENT
DATA1	DB	23H	;define DATA1 as a byte of 23H
DATA2	DW	1000H	;define DATA2 as a word of 1000H
START:	MOV	AL,BL	;copy BL into AL
	MOV	BH,AL	;copy AL into BH
	MOV	CX,200	;copy 200 decimal into CX

.MODEL	TINY		.MODEL TINY	;choose single segment model
.CODE		0000	.CODE	;start of code segment
.STARTUP			.STARTUP	;start of program
MOV	AX,0			
MOV	BX,0	0100 B8 0000	MOV AX,0	;place 0000H into AX
MOV	CX,0	0103 BB 0000	MOV BX,0	;place 0000H into BX
		0106 B9 0000	MOV CX,0	;place 0000H into CX
MOV	SI,AX			
MOV	DI,AX	0109 8B F0	MOV SI,AX	;copy AX into SI
MOV	BP,AX	010B 8B F8	MOV DI,AX	;copy AX into DI
		010D 8B E8	MOV BP,AX	;copy AX into BP
.EXIT			.EXIT	;exit to DOS
END			END	;end of program

.MODEL SMALL	64KB Code Memory and 64KB Data Memory
.MODEL MEDIUM	Code Memory can exceed 64KB but Data Memory remain 64KB
.MODEL COMPACT	Code Memory remain 64KB and Data Memory can exceed 64KB of Memory
.MODEL LARGE	Both Code and Data Memory can exceed 64KB of Memory, but no single set of data should exceed 64KB
.MODEL HUGE	Both Code and Data Memory can exceed 64KB of Memory, Data can exceed 64KB
.MODEL TINY	Used with COM files, in which code and data must fit into 64KB.

Data Types and Directives

- ORG (origin)

ORG is used to indicate the beginning of the offset address. The number that comes after ORG can be either in hex or in decimal. If the number is not followed by H, it is decimal and the assembler will convert it to hex.

- DB (define byte)

It allows allocation of memory in byte-sized chunks. This is indeed the smallest allocation unit permitted. DB can be used to define numbers in decimal, binary, hex, and ASCII.

```
0000 19          DATA1 DB 25          ;DECIMAL
0001 89          DATA2 DB 10001001B   ;BINARY
0002 12          DATA3 DB 12H         ;HEX
0010           ORG 0010H
0010 32 35 39 31 DATA4 DB '259'       ;ASCII NUMBERS
0018           ORG 0018H
0018 00          DATA5 DB ?          ;SET ASIDE A BYTE
0020           ORG 0020H
0020 4D 79 20 6E 61 6D DATA6 DB 'My name is Joe' ;ASCII CHARACTERS
      65 20 69 73 20 4A
      6F 65
```

0018H	00	0031H	
0017H		0030H	
0016H		002FH	
0015H		002EH	
0014H		002DH	65
0013H		002CH	6F
0012H	39	002BH	4A
0011H	35	002AH	20
0010H	32	0029H	73
000FH		0028H	69
000EH		0027H	20
000DH		0026H	65
000CH		0025H	6D
000BH		0024H	61
000AH		0023H	6E
0009H		0022H	20
0008H		0021H	79
0007H		0020H	4D
0006H		001FH	
0005H		001EH	
0004H		001DH	
0003H		001CH	
0002H	12	001BH	
0001H	89	001AH	
0000H	19	0019H	

Data Types and Directives

- DUP (duplicate)

DUP is used to duplicate a given number of characters. This can avoid a lot of typing. For example, contrast the following two methods of filling six memory locations with FFH.

0030		ORG	0030H	
0030	FF FF FF FF FF FF	DATA7 DB	0FFH,0FFH,0FFH,0FFH,0FFH,0FFH	; 6 BYTES = FF
0038		ORG	38H	
0038	0006[	DATA8 DB	6 DUP(0FFH)	;FILL 6 BYTES WITH FF
	FF			
	]			
0040		ORG	40H	
0040	0020 [	DATA9 DB	32 DUP (?)	;SET ASIDE 32 BYTES
	??			
	]			
0060		ORG	60H	
0060	0005[	DATA10 DB	5 DUP (2 DUP (99))	;FILL 10 BYTES WITH 99
	0002[			
	63			
	]			

- DW (define word)

DW is used to allocate memory 2 bytes (one word) at a time. DW is used widely in the 8088/8086 and 80286 microprocessors since the registers are 16 bits wide.

0070		ORG	70H	
0070	03BA	DATA11 DW	954	;DECIMAL
0072	0954	DATA12 DW	100101010100B	;BINARY
0074	253F	DATA13 DW	253FH	;HEX
0078		ORG	78H	
0078	0009 0002 0007 000C	DATA14 DW	9,2,7,0CH,00100000B,5,'HI'	;MISC. DATA
	0020 0005 4849			
0086	0008[	DATA15 DW	8 DUP (?)	;SET ASIDE 8 WORDS
	????			
	]			

Data Types and Directives

- EQU (equate)

This is used to define a constant without occupying a memory location. EQU does not set aside storage for a data item but associates a constant value with a data label so that when the label appears in the program, its constant value will be substituted for the label.

```
COUNT      EQU      25
COUNTER1   DB       COUNT
COUNTER2   DB       COUNT
```

- DD (define doubleword)

The DD directive is used to allocate memory locations that are 4 bytes (two words) in size.

```
00A0      ORG 00A0H
00A0 000003FF      DATA16 DD 1023      ;DECIMAL
00A4 0008965C      DATA17 DD 10001001011001011100B ;BINARY
00A8 5C2A57F2      DATA18 DD 5C2A57F2H      ;HEX
00AC 00000023 00034789 DATA19 DD 23H,34789H,65533
```

- DQ (define quadword)

DQ is used to allocate memory 8 bytes (four words) in size. This can be used to represent any variable up to 64 bits wide.

```
00C0      ORG 00C0H
00C0 C223450000000000 DATA20 DQ 4523C2H      ;HEX
00C8 4948000000000000 DATA21 DQ 'HI'          ;ASCII CHARACTERS
00D0 0000000000000000 DATA22 DQ ?              ;NOTHING
```

- DT (define ten bytes)

DT is used for memory allocation of packed BCD numbers.

```
.MODEL SMALL
.DATA
```

```
.CODE
.STARTUP
```

```
.EXIT
END
```

0000		.MODEL SMALL	;choose small model
		.DATA	;start data segment
0000 10	DATA1 DB 10H		;place 10H into DATA1
0001 00	DATA2 DB 0		;place 00H into DATA2
0002 0000	DATA3 DW 0		;place 0000H into DATA3
0004 AAAA	DATA4 DW 0AAAAH		;place AAAAH into DATA4
0000	.CODE		;start code segment
	.STARTUP		;start program
0017 A00000 R	MOV AL,DATA1		;copy DATA1 into AL
001A 8A 26 0001 R	MOV AH,DATA2		;copy DATA2 into AH
001E A3 0002 R	MOV DATA3,AX		;copy AX into DATA3
0021 8B 1E 0004 R	MOV BX,DATA4		;copy DATA4 into BX
	.EXIT;		exit to DOS
	END;		end program listing

;SIMPLIFIED FORMAT

```
.MODEL SMALL
.STACK 64
```

•
•
•
•
•
•

. DATA

```
;data definitions are placed here
```

•
•
•
•

```
.CODE
```

```

MAIN  PROC FAR
      MOV  AX,@DATA
      MOV  DS,AX
      ...

```

```

...
MAIN ENDP
      END MAIN

```

• •

• •

MAIN ENDP

END MAIN

■ ■ ■

```
MAIN ENDP
```

name3 ENDS

END MAIN


```

.MODEL SMALL
.DATA
ARRAY DB 16 DUP(?)

DB 29H
DB 20 dup(?)

.CODE
.STARTUP

MOV BX,OFFSET ARRAY
MOV DI,10H
MOV AL,[BX+DI]
MOV DI,20H
MOV [BX+DI],AL

.EXIT
END

```

```

.MODEL SMALL
.DATA
ARRAY DB 16 dup(?)

DB 29
DB 30 dup(?)

.CODE
.STARTUP
MOV DI,10H
MOV AL,ARRAY[DI]
MOV DI,20H
MOV ARRAY[DI],AL
.EXIT
END

```